

Does a Symbolic Data-Movement Analyzer Help an LLM Optimize Loop Kernels?

A controlled multi-agent study of the AutoLALA dmd tool

AutoLALA experiment harness — assignments-test/

2026-06-25

Summary

We ran a controlled A/B experiment to measure whether giving a Claude agent the AutoLALA dmd locality analyzer makes it a better and/or cheaper optimizer of affine loop kernels. Agents work in matched **tool** (analyzer allowed) vs **control** (analyzer forbidden) pairs on the same kernels; the only difference between an agent and its partner is access to the tool.

The headline finding comes from the **final run**, in which agents write **MLIR affine** loop nests (so the only optimization lever is loop *structure / locality* — exactly what dmd models) and are graded on **real single-core wall-clock runtime**:

The tool group reached a **geometric-mean average speedup of 3.50x** vs the control group's **3.29x** (a **+6%** edge), but the per-kernel head-to-head was a literal **3-3 tie**, and the tool group spent **1.6-2.5x more output tokens** to get there. All 12 submissions were numerically correct.

In other words: when the task is confined to the locality decisions the analyzer is built for, the tool's signal is real but *marginal*, and not obviously worth its token premium. Two earlier runs (described in §6) bracket this result — when the metric is the tool's own data-movement model the tool wins trivially; when agents may write raw C the control group wins because C exposes register/SIMD/ILP levers the model cannot see.

1. Background: what the dmd tool is

AutoLALA (autodmd) is a Rust workspace for **symbolic data-movement complexity (DMD) analysis** of affine loop-tree programs. Given an affine loop nest (as MLIR or its small DSL), it lowers the program into polyhedral sets/maps via Barvinok, computes reuse-interval and reuse-distance distributions, and emits a symbolic DMD formula — a closed-form estimate of memory traffic as a function of the loop bounds and a cache block size.

It is exposed to an agent as a read-only MCP server (dmd-mcp) with five advisor tools: `analyze_mlir`, `analyze_dsl`, `validate_dsl`, `extract_from_mlir`, and `compare_variants`. The premise under test: *does this locality model actually help an agent make better optimization decisions?*

2. The research question

Holding the agent, the kernel, and the task fixed, does access to the dmd analyzer improve the quality and/or the token-efficiency of the loop optimizations an agent produces — when “quality” is **measured runtime**, not the model’s own metric?

This requires three things the naive setup lacks: (a) a ground truth independent of the tool, (b) a task where the tool’s domain (locality) is the deciding factor rather than a confound, and (c) a clean manipulation. The final run supplies all three.

3. Experiment setup (final run)

3.1 Why MLIR-affine instead of C

An earlier run let agents optimize **C**. The result was dominated by hand-vectorization, restrict/aliasing hints, and register blocking — levers that have nothing to do with the dmd model and that a strong coder exploits directly. To *isolate the locality dimension*, the final run constrains agents to write **MLIR affine** loop nests:

- Allowed dialects: affine, arith, memref, func. The agent expresses only the loop **structure** (order, tiling, fusion, unroll-and-jam), not machine code.
- The coordinator (not the agent) compiles every submission with a **fixed** pipeline, so low-level vectorization is identical for all agents and is *not* something they can hand-tune:

```
mlir-opt opt.mlir -lower-affine -convert-scf-to-cf -convert-cf-to-llvm \
  -convert-arith-to-llvm -finalize-memref-to-llvm -convert-func-to-llvm \
  -reconcile-unrealized-casts \
  | mlir-translate --mlir-to-llvmir \
  | llc -O3 -filetype=obj -o opt.o          # llc -O3 does the same vectorization for all
clang -O3 -march=native driver.c opt.o -o bin
```

Each kernel has a fixed C-interface signature (`func.func @kernel(... memref ...)` { `llvm.emit_c_interface` }). The coordinator writes the **C driver** that builds the memref descriptor structs, fills the arrays with a deterministic xorshift PRNG, calls `_mlir_ciface_kernel`, and dumps the output array(s) as raw float64. Agents never touch the driver. (Measured on this aarch64 host, `llc -O3` does **not** auto-vectorize these strict-FP reduction loops, so the realized lever is genuinely memory traffic + scalar ILP — precisely the locality regime.)

3.2 Kernels

Six PolyBench-style kernels spanning two locality regimes:

kernel	computation	dominant locality issue
matmul	$C += A \cdot B$	$O(N^3)$ reuse; classic tiling
gemm	$C = 0.9 \cdot C + A \cdot B$	tiling + a scaling pass
2mm	$T = A \cdot B$; $D = 0.9 \cdot D + T \cdot C$	chained MM; tiling + fusion
mvt	$x1 += A \cdot y1$; $x2 += A^T \cdot y2$	transposed sweep (interchange/fusion)
atax	$T = A \cdot x$; $y = A^T \cdot T$	two A-sweeps (fusion)
syrk	$C = 0.9 \cdot C + A \cdot A^T$, lower-tri	triangular tiling

The first three are compute-bound $O(N^3)$ (large tiling headroom); the last three are bandwidth-bound BLAS-2 (interchange/fusion headroom).

3.3 The manipulation: tool vs control

12 agents = 6 kernels x {tool, ctrl}. All are the same base claude agent with an identical performance-only task. The **only** difference:

- **tool**: *may* consult dmd (MCP tools or CLI). Told explicitly to treat its output **only as a directional hint** about the data-movement trend — not a runtime predictor.
- **ctrl**: *forbidden* from any dmd/mlir-extract/dmd-cli access; optimizes from first-principles knowledge of caches and affine-loop theory.

Both groups may compile and run `check.sh` to verify **correctness**, but **neither may benchmark/time** its code — so the tool is the sole asymmetry. Compliance was verified from agent transcripts (control agents made zero dmd calls).

3.4 The three size regimes

Each kernel is graded at **small / medium / large**, each a *range* with the exact test N hidden from the agent, so a kernel cannot be over-fit to one size:

kernel	small (range -> testN)	medium	large
matmul, gemm	[192,384]->256	[512,1024]->768	[1152,1536]->1280
syrk	[192,384]->256	[512,1024]->768	[1152,1664]->1408
2mm	[160,320]->224	[448,768]->576	[896,1152]->1024
mvt, atax	[1024,2048]->1536	[3072,5120]->4096	[6144,8192]->7168

Agents may submit a single `opt.mlir` or three tuned versions (`opt_small/medium/large.mlir`). A structure that wins at large N can lose at small N (tiling overhead when the working set already fits in cache), so the three regimes reward size-aware reasoning. **Score = arithmetic mean of the three regime speedups.**

3.5 Grading (ground truth)

For each submission and regime (`batch_eval.py`):

1. **Build:** lower the agent's MLIR -> native object, link the driver.
2. **Correctness:** dump the full output array(s) at $N = 96$ and $N = 130$ (a non-tile multiple) and require `numpy.allclose(ref, opt, rtol=1e-6, atol=1e-9)` against the reference kernel. This is a *full-array* check, not a checksum.
3. **Performance:** hyperfine, warmup 1 + 4 runs, pinned to one core (`taskset -c 0`), at the regime's hidden test N . **speedup = $t_{\text{ref}} / t_{\text{opt}}$.**

A submission must pass correctness at *every* regime to be scored.

4. Results (final run)

All 12 submissions built and were numerically correct at every regime, including non-tile-multiple N — **12/12 correct, 0 regressions** in both groups.

4.1 Average speedup vs the naive affine reference

kernel	tool avg	ctrl avg	winner	margin
matmul	4.43	6.12	ctrl	+38%
gemm	5.86	4.28	tool	+37%
2mm	4.26	4.39	ctrl	+3%
mvt	2.62	1.67	tool	+57%
atax	1.48	1.64	ctrl	+11%
syrk	4.29	4.01	tool	+7%

4.2 Per-regime detail (speedup small / medium / large)

kernel	tool (s/m/l)	ctrl (s/m/l)
matmul	3.74 / 4.30 / 5.25	4.96 / 6.02 / 7.38
gemm	4.91 / 5.74 / 6.92	3.73 / 4.31 / 4.80
2mm	3.33 / 3.85 / 5.59	3.41 / 3.98 / 5.77
mvt	2.18 / 2.67 / 3.00	1.45 / 1.70 / 1.86
atax	1.34 / 1.40 / 1.69	1.55 / 1.49 / 1.88
syrk	3.90 / 4.80 / 4.18	3.23 / 4.41 / 4.37

Every kernel’s speedup grows from small->large, confirming the transformations attack capacity misses that worsen with N — i.e. the regimes behave as designed.

4.3 Aggregate

metric	TOOL	CTRL	TOOL / CTRL
geometric-mean avg-speedup	3.50	3.29	1.06
mean avg-speedup	3.82	3.69	1.04
head-to-head wins (>3% margin)	3	3	tie
correct / 6	6	6	—
output tokens (contestant only)	149,760	92,369	1.62x
output tokens (incl. analyzer sub-agents)	232,404	92,369	2.52x

5. Analysis

The tool’s advantage is real but small, and concentrated where locality is the whole story. The tool group’s wins — gemm, mvt, syrk — are exactly the kernels whose optimum is

a *structural* call the analyzer is good at surfacing: fuse the two passes, kill the transposed A^T sweep by interchange, choose i-k-j. On mvt the hint pushed the tool agent to *fuse* both matrix-vector products so A streams once (2.62x vs the control’s interchange-only 1.67x).

But the tool does not dominate, and lost its biggest matchup, because of a lever it cannot see. On matmul the control agent won decisively (6.12x vs 4.43x) by adding **unroll-and-jam x4** on top of tiling — exposing four independent FMA chains the scalar back-end pipelines. The tool agent, anchored on the analyzer’s data-movement framing, tiled correctly but under-exploited that register-level ILP. The dmd model quantifies *how much data moves*, not *how well the loop body keeps the pipeline busy* — and even in restricted affine MLIR, the latter decided a kernel.

The token premium is consistent and not repaid. Tool agents spent 1.62x more tokens on their own, and 2.52x once the analyzer sub-agents they spawned are counted, for a 6% geomean gain and an even head-to-head. The analyzer’s symbolic output is verbose and the agents that leaned on it tended to iterate more.

6. The three-experiment arc

This final run is the third and most controlled of three experiments in this directory, which together explain *why the ground truth matters*:

#	ground truth	what agents wrote	winner	why
1	symbolic DMD (the tool's <i>own</i> model)	MLIR	tool (geo 8.5 vs 6.8)	tautological: scored on exactly what the tool optimizes
2	real runtime	C	control (geo 5.9 vs 5.6; 6-3 H2H)	C exposes SIMD/register/ILP the model can't see; 1 tool regression
3	real runtime	MLIR affine (this run)	~ = tie , tool +6%	locality isolated -> the tool's signal finally shows, but only marginally

The progression is the point. Experiment 1 is the trap: evaluate a tool on its own metric and it always wins. Experiment 2 shows that against *real* runtime in an unconstrained setting the tool actively loses, because performance is mostly decided outside its model. Experiment 3 — the fairest test for the tool — removes those confounds and finds the tool's genuine contribution: a small, real edge on the structural/locality decision, worth roughly a coin flip head-to-head and a token premium.

7. Threats to validity

- **Host = aarch64, single machine.** Absolute speedups and the exact tiling sweet-spots are hardware-specific; the *direction* of the tool-vs-control comparison is the durable result, not the constants.
- **llc -O3, not a production tiler.** A different back-end (e.g. one that auto-tiles) would compress all speedups and could change margins.
- **n = 6 kernels, 1 agent per cell.** Head-to-head 3-3 is within noise; the geomean and the cross-experiment trend are the load-bearing claims, not any single kernel.
- **MCP reachability.** The dmd MCP server was not always reachable inside sub-agents, so tool agents often used the identical engine via its CLI fallback. The manipulation (tool

measured locality, control did not) held; the token accounting includes the delegated analyzer sub-agents.

- **“No benchmarking” is instruction-enforced.** Transcripts were checked, but the bar is honor-system; a determined agent could have timed its own code.

8. Conclusion and recommendation

A symbolic data-movement analyzer is a **genuine but narrow** aid. Constrained to the locality decisions it models — loop order, tiling, fusion — it roughly matches a strong unaided optimizer and occasionally finds a structural win the unaided agent misses (mvt, gemm). It is **blind** to the register/ILP/SIMD levers that frequently decide real single-core runtime (matmul), and it carries a **1.6-2.5x token premium**.

Recommendation: use dmd as a cheap, directional sanity check on loop *structure* early in optimization — “should I tile? fuse? interchange?” — and then **measure runtime** for the final decision. Do not treat its data-movement numbers as a performance oracle, and do not let it be the optimizer of record.

Appendix: reproduction

```
assignments-test/
+-- mlirperf/                                # the final (MLIR-affine) run
|   +-- generate.py                          # emits ref.mlir + driver.c + spec.md per kernel
|   +-- lower.sh                            # MLIR affine -> native object (llvm-22)
|   +-- batch_eval.py                      # correctness (allclose) + hyperfine grading
|   +-- kernels/<k>/                        # ref.mlir, driver.c, spec.md
|   +-- runs/<group>-<k>/                   # each agent's opt*.mlir + rationale.md
|   +-- results.json                       # raw per-regime times + speedups
|   `-- mlir_metrics.json                  # per-kernel + aggregate metrics
+-- perf/ RESULTS_PERF.md                  # experiment 2 (C, real runtime)
`-- RESULTS.md                             # experiment 1 (symbolic DMD metric)
```

Regenerate the final run: `python3 generate.py && python3 batch_eval.py` (grading lowers every submission with llvm-22, links the fixed driver, checks `numpy.allclose`, and times with hyperfine pinned to one core).